[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

1. Introduction, motivation, metaheuristics, module structure and some evolution in nature

University of Essex
School of Computer Science and Electronic Engineering

Outline

- **Hands-on experience:** hints for knapsack problem
- **Why** do we need evolutionary computation and genetic programming?
- **What** is in the module and **how** will it be run?
- **How** does natural evolution work?

Outline

- **Who** is your module supervisor?
- **Why** do we need evolutionary computation and genetic programming?
- How will the model be run? (Introduction to CE310)
- Evolution in nature

Module supervisor

Riccardo Poli

- Professor in Computer Science
- Expert in Evolutionary Computation and Genetic Programming
- Co-founder and former co-director Essex BCI-NE lab

Email: rpoli@essex.ac.uk

Office: 1NW.5.3B

Three decades of experience in EAs and GP

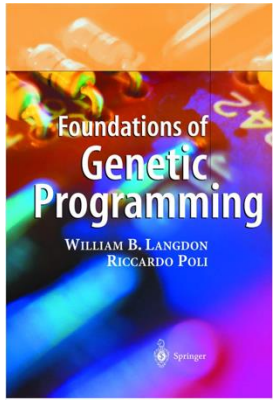
- First publication on EAs/GP 1993
- Involved in many EA/GP conferences and journals in the past
- ISGEC Fellow (2003), EvoStar award (2007) and ACM SigEvo award (2023).
- Fourth (out of 19,000+) most prolific author in GP worldwide (253 articles, 113 first author)
- 2002 first book on the theory of GP (with B. Langdon)
- 2008 field guide to GP book (with B. Langdon, N. McPhee and J. Koza,one of the most cited books in GP). **Free!!**
- Recently evolving better learning rules for neural networks using GP and completing the book "Taming the Complexity of Evolutionary Dynamics" (with C. Stephens)

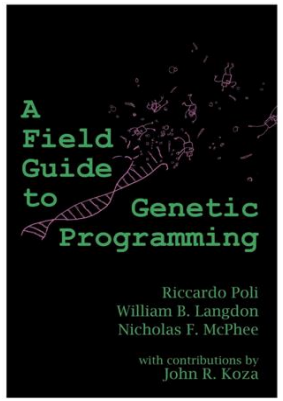
Genetic Programming Bibliography

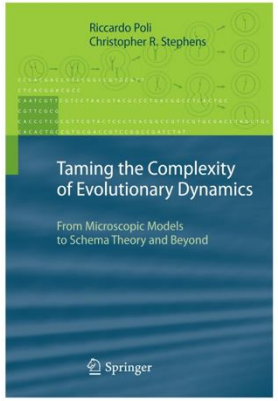
To add references to your papers look here or send details to me by email.

Number of genetic programming entries by author (total) in weighted order

1. 260 (347) William B Langdon
2. 42 (669) Mengjie Zhang
3. 171 (208) John Koza
4. 113 (253) Riccardo Poli
5. 44 (276) Wolfgang Banzhaf
6. 58 (299) Michael O'Neill
7. 67 (161) Lee Spector
8. 43 (215) Conor Ryan
9. 43 (143) Julian F Miller
10. 52 (121) Krzysztof Krawiec







Outline

- **Who** is your module supervisor?
- **Why** do we need evolutionary computation and genetic programming?
- **How** will the module be run? (**Introduction to CE310**)
- **How** does natural evolution work?

Let’s start with an example: Relocation

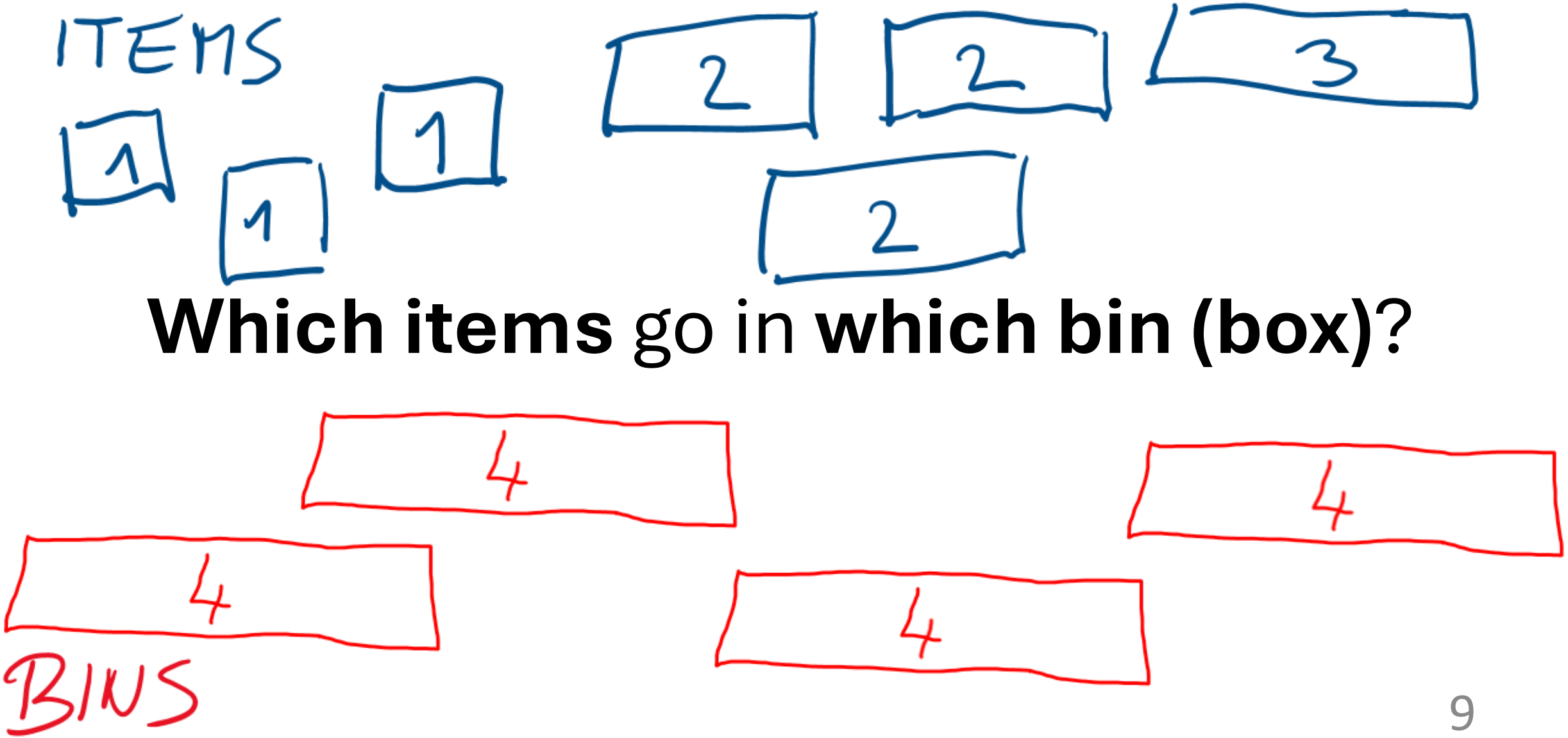
- **Problem 1: Which** of your **things** go in **which box** so that you have as **few boxes** as possible?
- Boxes are expensive, so with fewer boxes you save money
- Also, if you need to send the boxes via currier, fewer boxes → cheaper delivery

Picture from https://www.apartmentshere.com/top-10-moving-tips/

Problem 1: Bin Packing Problem (idealized, 1-D)

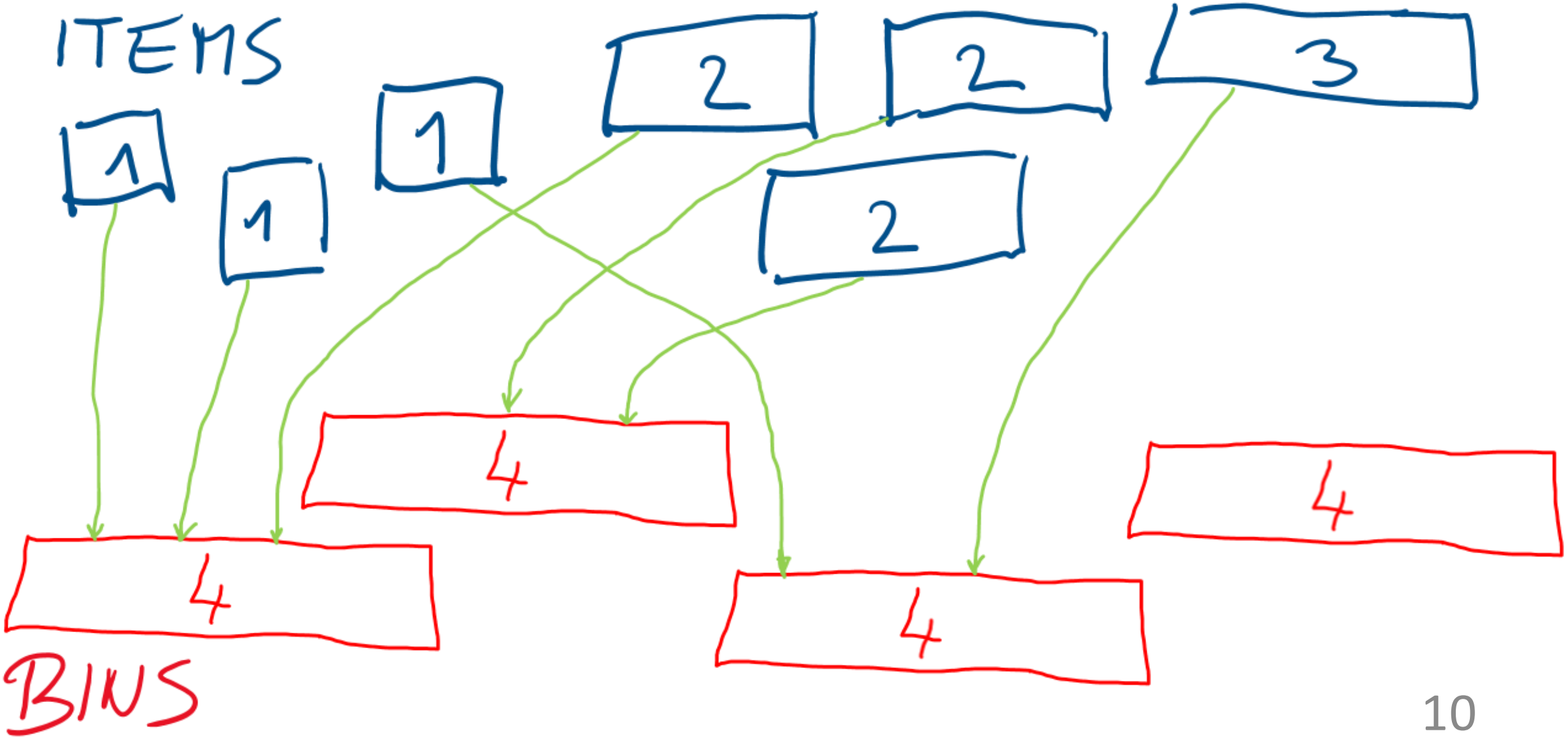
- Given are **N items** of different **sizes s_i** with **i=1…N** and **B bins** (boxes) each of **capacity c**.
- Goal is to assign each item to a bin such that the total number bins is *minimised*. Let's assume that all items have **sizes s_i <= c**.
- **Why relevant?** Applications include loading goods in shipping containers, warehouse storage, placing files on multiple disks, timetabling, …

Example Problem 1

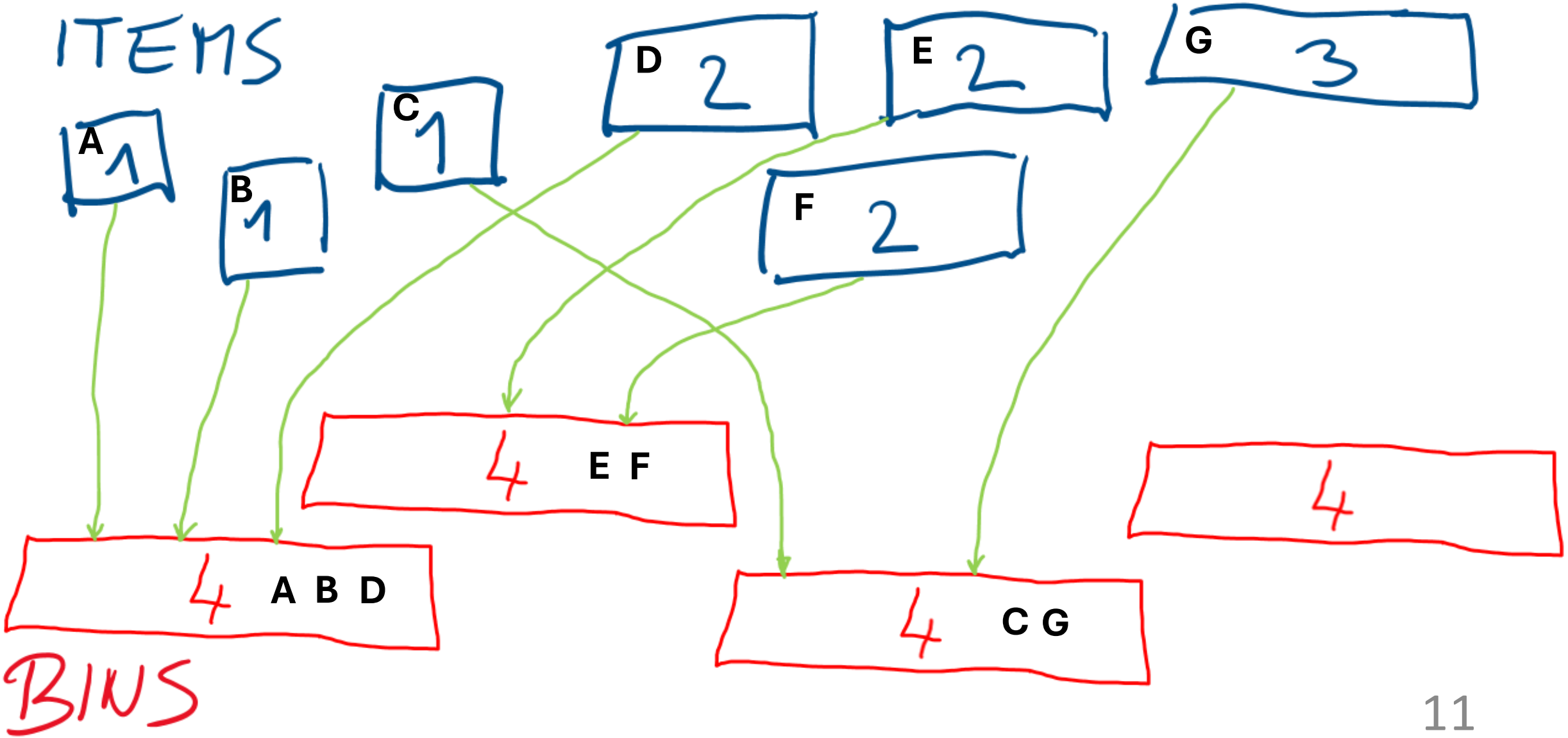


Which items go in **which bin (box)**?

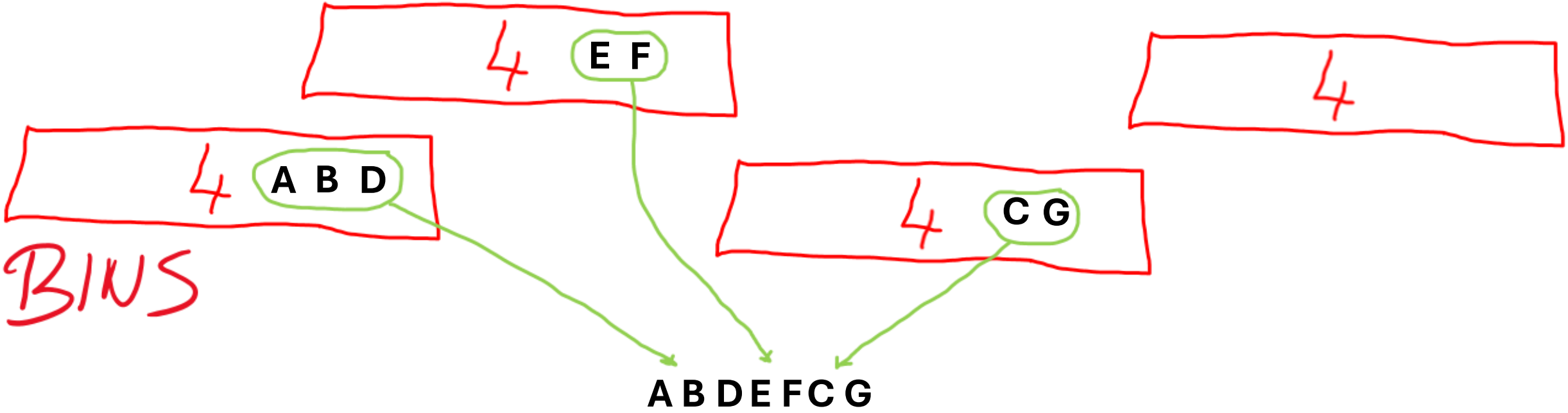
Example Problem 1: Intuitive solution (1)



Example Problem 1: Intuitive solution (2)



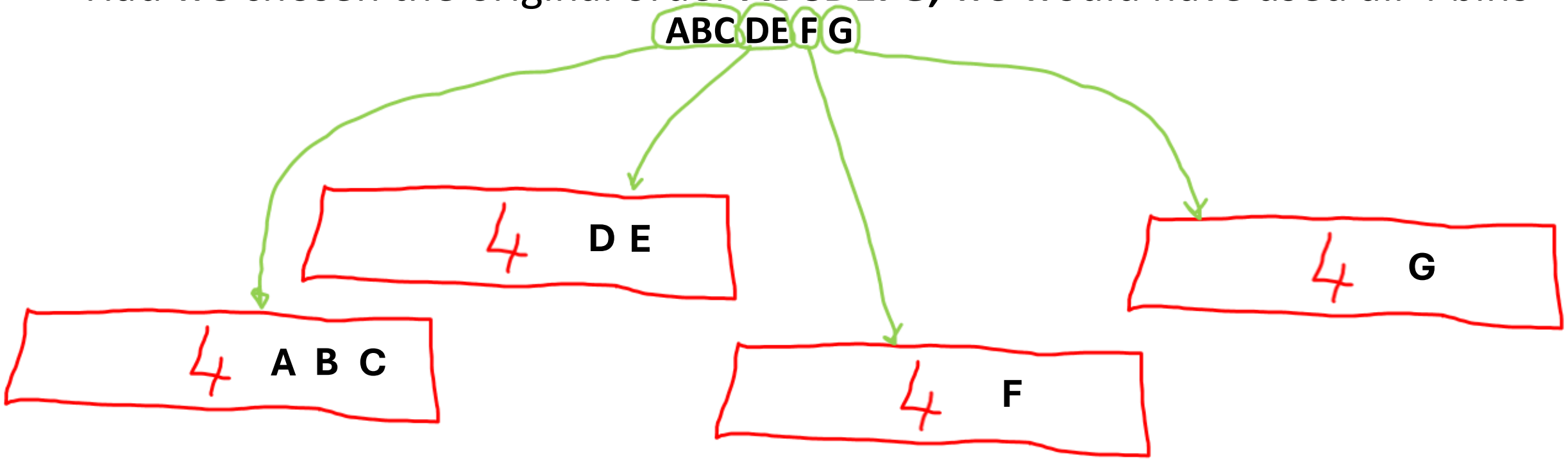
Example Problem 1: Intuitive solution (3)



- So, if we put the items **ABCDEFG** in bins one at a time in the order **ABDEFCG** we end up using only 3 bins out of 4

Example Problem 1: Intuitive solution (4)

- **ABDEFCG** is called a *permutation* of **ABCDEFG** (permutation=reshuffling)
- Had we chosen the original order **ABCDEFG,** we would have used all 4 bins



- But how do we know the optimal order/permutation?
- **A "brute-force" approach to solve the problem: Test all permutations!**

How many permutations are there? (1)

- Suppose we have 3 items to place in boxes: **A**, **B** and **C**
  - There are **three ways** of placing **A**:

    A can be first (**A** _ _), second (_ **A** _), or third (_ _ **A**).
  - For every placement of **A**, we only have **two ways** available for placing **B**. E.g., (A **B** _) and (A _ **B**).
  - Once both **A** and **B** are placed, there is only **one slot** left for **C** irrespective of where I placed **A** and **B**.
  - So, in total there are 3 x 2 x 1 = 6 = 3! ways of permuting A, B and C:

    ABC, ACB, BAC, BCA, CAB and CBA

! FACTORIAL

How many permutations are there? (2)

- In general, the number of permutations of *n* elements is

$$P_n = n!$$

- In our example bin packing problem, we have *n*=7

$$P_7 = 7! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 = 5{,}040$$

Combinatorial Optimization

- Problem 1 is a <u>combinatorial optimisation problem</u>.
- Combinatorial Optimization is concerned with finding an **optimal** or **close to optimal solution** among a finite collection of possibilities
- Applications: Logistics, Travel route, Timetabling, Disk partitioning, Hyper-parameter selection for AI/ML, …

Another combinatorial optimisation problem: relocation example 2

- **Problem 2: Your things have different values, and you cannot fit them all in your car. Which items should you take with you in the first trip to your new home?**
- You want to load the car with as many of the *most valuable* items as possible

Picture from https://d2y7gcw37cyxhu.cloudfront.net/images/layout/cat_home/cat_umzugstransporter.jpg

Problem 2: Knapsack (Rucksack) problem

- Problem you face if you are constrained by a fixed-size knapsack and must fill it with the most valuable items.
- Given a **set of items** (i=1…N), each with a **size** (s_i) and a **value** (v_i), determine which items to include in the knapsack so that the **total size is less than or equal to a given limit** (S_{max}) and the **total value is as large as possible** (***maximize*** $\mathbf{v_{tot}}$).

For more info see https://en.wikipedia.org/wiki/Knapsack_problem

Computational complexity theory

- What is the **computational complexity** (amount of time needed to run) of the developed solution?
- Testing **all** permutations leads to the optimal solution, but the time required may be too high!
- The number of operations required grows at least exponentially **O(e^n)** with size of input **n** (*e*= 2.718..)

$$n! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 \ldots > 1 \cdot 2 \cdot e \cdot e \cdot e \cdot e \cdot e \cdot \ldots$$

$$n! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 \cdot 8 \cdot 9 \cdot 10 \cdot 11 \cdot 12 \ldots > 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 \cdot 8 \cdot 9 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot 10 \cdot \ldots$$

- This is the NP (**N**ondeterministic **P**olynomial) time complexity class. Most combinatorial optimisation problems are NP!
- The exponential growth in possible solutions = *combinatorial explosion*
- So, what can we do?

Meta-heuristic stochastic optimization

- **Heuristic** (from Greek heuriskein = to search): Approximate strategies (rules of thumb) or partial search algorithm which are *problem specific*.
- **Metaheuristic** (metá = beyond): Higher-level, more *general* procedure designed to find, generate, or select a heuristic that may provide a sufficiently good solution to an optimization problem, with an acceptable computation load
- **Stochastic**: Randomly determined process
- **EAs and GP are highly successful meta-heuristics suitable for combinatorial optimization.**

Questions?

What have we learnt so far today?

- We have learned about **combinatorial optimisation** problems (Bin Packing problem & Knapsack problem)
- We have seen that **exact/complete algorithms** can be **computationally complex** (especially intuitive solutions).
- Consequently, such algorithms may not be a viable solution to real-world problems, and we need alternatives (meta-heuristics).

Outline

- **Hands-on experience:** hints for knapsack problem
- **Why** do we need evolutionary computation and genetic programming?
- **What** is in the module and **how** will it be run?
- **How** does natural evolution work?

What is in the module and **how** will it be run?

- The module is all about **optimisation** and **problem solving**, so we are in the AI domain

- Elements in this section:
  - What is **Evolutionary Computation**?
  - What is **Genetic Programming**?
  - Course description, outcomes, assessment
  - Sources of information (including online)

Evolutionary Computation

- Evolutionary Computation is a branch of Computer Science that studies and applies **evolutionary algorithms** (**EAs**)
- EAs are optimization and search procedures inspired by **natural evolution**.
- Depending on the structures undergoing optimization, the **reproduction strategies** and the **genetic operations** EAs can be grouped into: **Genetic Algorithms** (GAs), **Evolution Strategies** (ESs), **Genetic Programming** (GP), ...

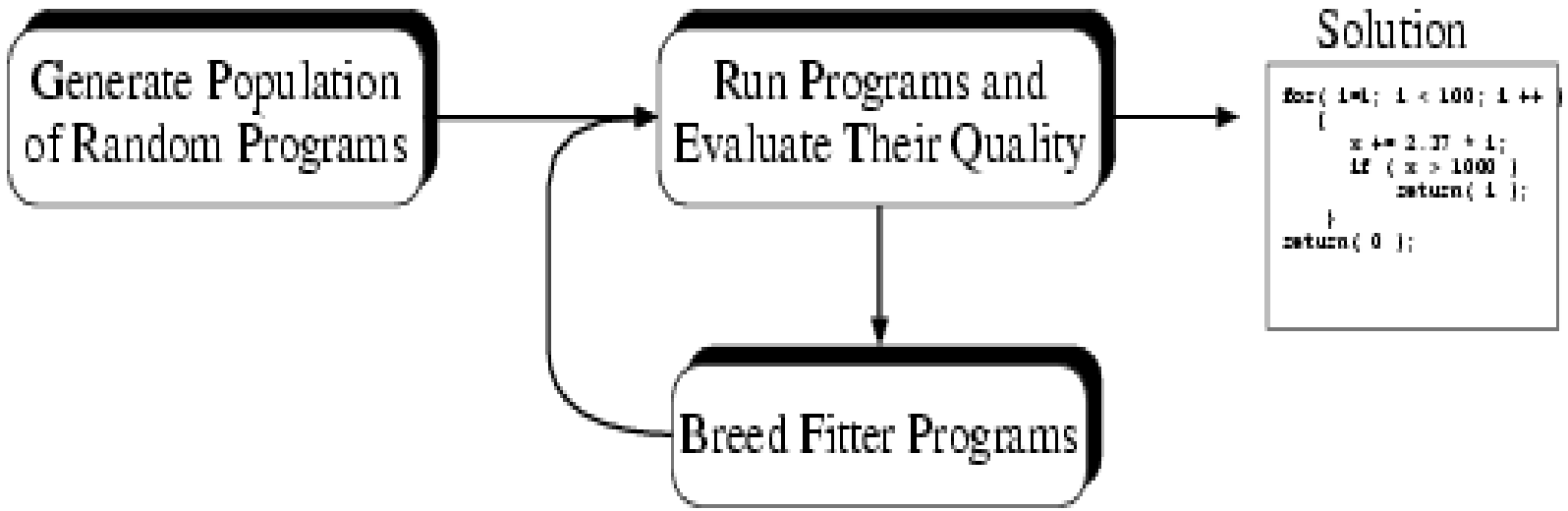
Evolutionary Algorithms

- EAs are systematic methods inspired by *Darwinian evolution* for **getting computers to automatically solve problems** starting from a high-level statement of what needs to be done
- EAs are often domain-independent methods (i.e., they can be applied to a variety of domains with relatively little effort) → Need to only slightly adjust code
- GAs and ESs are different flavours of EA which act on **linear representations** (arrays) (e.g. problems 1 & 2)

Genetic Programming

- Genetic Programming (GP) is a class of EAs for solving problems which require solutions in the form of **computer programs**.
- GP genetically breeds a population of computer programs to solve a problem.
- EAs and GP iteratively transform a population of *random* solutions into new generations of better and better solutions.

Genetic Programming



- EAs and GP apply analogues of genetic operations like survival of the fittest, sexual recombination, mutation, etc.

Course Description

- The course starts with a primer on natural evolution (today!) and introducing the basics of evolutionary algorithms (ESs and GAs, in the next 2-3 weeks).
- Then we move onto the basics of Genetic Programming (GP) later studying some of GP's more advanced variants.
- We touch upon other nature-inspired metaheuristics, such as Particle Swarm Optimisers (PSOs)
- Part of the course is also devoted to presenting real-world applications of EAs and GP.

...all accompanied by **hands-on experience** in the classes.

Learning Outcomes (1)

After completing this module, you can:

- Discuss Evolutionary Algorithms and their relationships.
- Describe Genetic Programming and its relationship with other Evolutionary Algorithms.

Learning Outcomes (2)

After completing this module, you can:

- Compare application domains for GP and associate these with good GP techniques.
- Identify GP parameters and modify existing GP operators, representations and fitness functions for specific applications.

Syllabus (1)

- Evolution in Nature
- Genetic Algorithms
- Evolution Strategies
- The basics of GP
- Fitness functions in GP
- Advanced Representations
- Code growth and methods to control it

Syllabus (2)

- Applications of GP
- Koza's criteria for human-competitive machine intelligence and review of GP's human-competitive results
- Advanced GP techniques and tricks of the trade
- Some other nature-inspired metaheuristics.

Delivery

- Normally, two hours of lectures per week to cover main body of the course and one hour class per week for formative exercises, problem solving, interactive coding, etc.
- **Learning by doing**: To gain insight into the functioning of the methods you are expected to **code core components yourself** and **experiment with the methods** outside of the lecture/class (spend 8 hours per week on the Module)

Assessment

- 60% Examination
- 40% Assignments (out around week 18 with deadline in week 23, see FASER)

Components:

1. Code and/or run EAs in Python to solve optimisation problems → 50%
2. Code and or run simulations to investigate the impact of GP parameters on performance → 50%
3. Possibly post-submission elements to verify understanding of code and experimental results

Academic integrity, authorship and plagiarism

- **Plagiarism**: “the act of using another person's words or ideas without giving credit to that person”
  [https://www.merriam-webster.com/dictionary/plagiarism]
- Intentional, reckless, or unintentional? Note that intentional or reckless plagiarism is a disciplinary offence
- **Please discuss the solutions to tasks among yourselves. But have the ambition to implement the solution yourself.**
- Questions? Please check https://www.essex.ac.uk/student/academic-skills/academic-integrity

Coursework help from AIs [work in progress]

- If you use AIs to help you with the coursework, you **must make sure you can answer any questions I may have on the code or analysis**.
- AIs tend to produce the same or similar answers over and over again and, so, *AI-generated code tends to be flagged up by our plagiarism detection system for code*.
- **Protocol** (v1) if you want to get help from AIs in producing the coursework:
  1. *Keep a log* of the steps below and *submit it with the coursework*
  2. Ask the AI to first produce the *smallest possible fragment of code* that gets you started,
  3. Make sure this does not only run but *you fully understand* it, e.g., you would be able to modify it by hand if required.

     2.1 if it is correct proceed to step 4,

     2.2 else fix any issues yourself (preferred) or ask the AI to fix (ok but make sure you fully understand the fix).

     2.3 If you cannot fully understand the code, you asked the AI to do too big a job. Ask for less.
  4. Ask the AI to do a *minimal extension of the code* that will take you closer to where it needs to be and repeat step 3 until you are satisfied.

Sources of Information: Main source

- Lectures and classes. Slides, videos, code and handouts available on Moodle
- R. Poli, W.B. Langdon, N.F. McPhee. **A field guide to Genetic Programming**, 2008.
  - Free PDF http://www.gp-field-guide.org.uk/

Sources of Information: Background

- S. Luke, **Essentials of Metaheuristics**, Lulu, Second edition, 2013.
  - Freely available as a PDF from here
- W.B. Langdon, R. Poli, Foundations of Genetic Programming, Springer, 2002.

Start with this book

Other On-line Resources (1)

- The Wikipedia (Evolutionary Algorithms) is a good starting point for online reading
  - http://en.wikipedia.org/wiki/Evolutionary_algorithms
- A **complete bibliography** on genetic programming maintained by Bill Langdon at
  - https://gpbib.cs.ucl.ac.uk/

Other On-line Resources (2)

- Use AIs to ask questions on genetic programming and evolutionary algorithms
- Google Scholar http://scholar.google.com

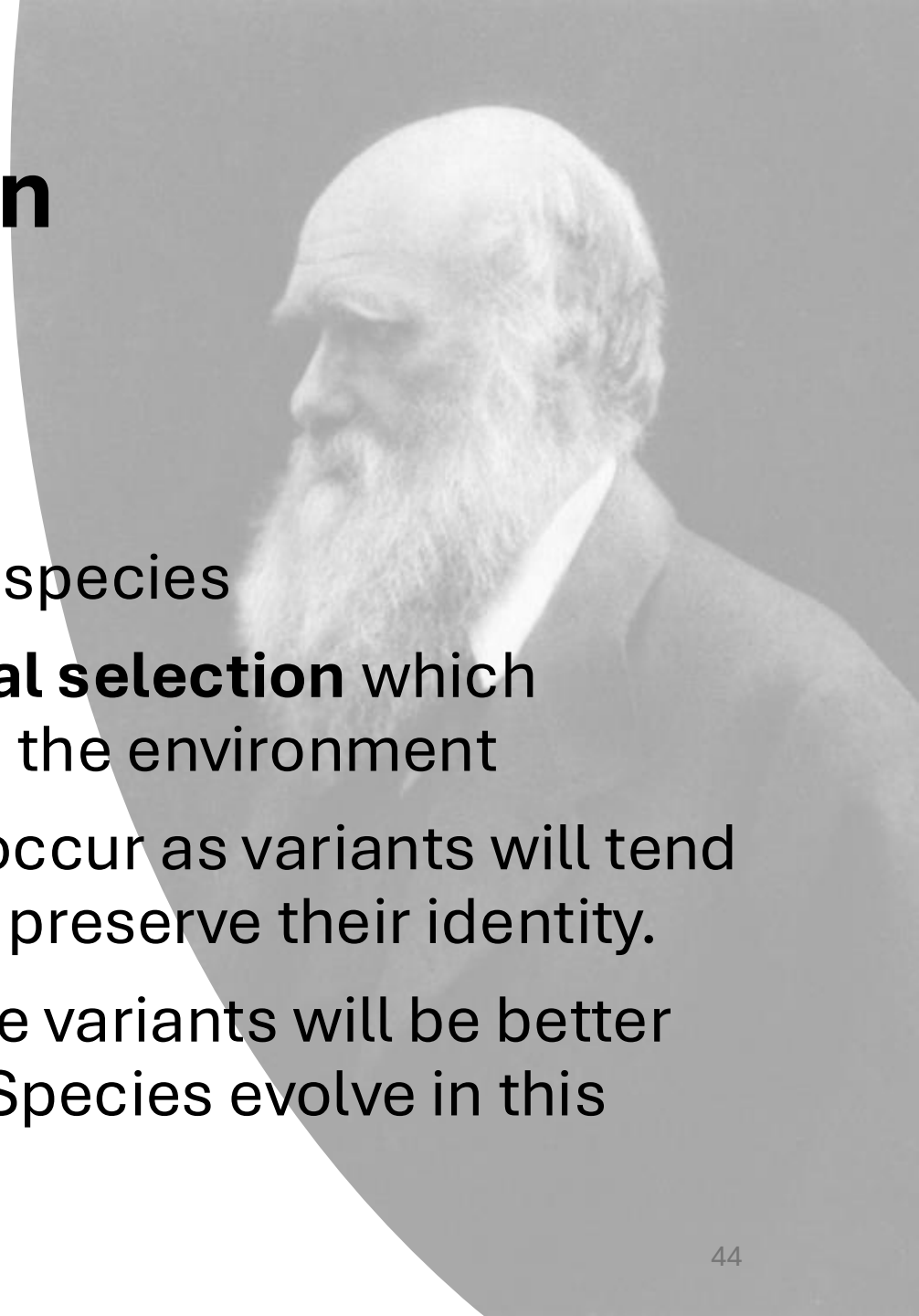
  365k hits for “genetic programming”, 613k hits for “evolutionary algorithms” and 2.1M hits for “genetic algorithm”

Questions on the module?

Outline

- **Hands-on experience:** hints for knapsack problem
- **Why** do we need evolutionary computation and genetic programming?
- **What is in the module and how will it be run?**
- **How** does natural evolution work?

Darwin's Theory of Evolution



- Charles Robert Darwin (1809-1882)
  - 1859: On the origin of species …
- **Variations** (**mutations**) are present in all species
- Evolution is due to a “force” called **natural selection** which “selects” the individuals best **adapted** to the environment
- In a **constant environment** no changes occur as variants will tend to lose in the struggle for life. So, species preserve their identity.
- In a **varying environment**, however, some variants will be better than the originals and will be preserved. Species evolve in this way.

Mendel's independent discoveries

- Gregor Johann Mendel (1822-1884)
  - 1865: Versuche über Pflanzenhybriden [Experiments on Plant Hybridization]
- Factors, what we now call **genes/alleles**, determine visible **traits** in **predictable** ways
- Dominant and recessive characters

Example: pea colour

P Generation

YY yy

F_1 Generation

Yy Yy

?

F_2 Generation

YY Yy Yy yy

?

https://www.futurelearn.com/info/courses/genomics-for-educators/0/steps/305264

How natural selection works

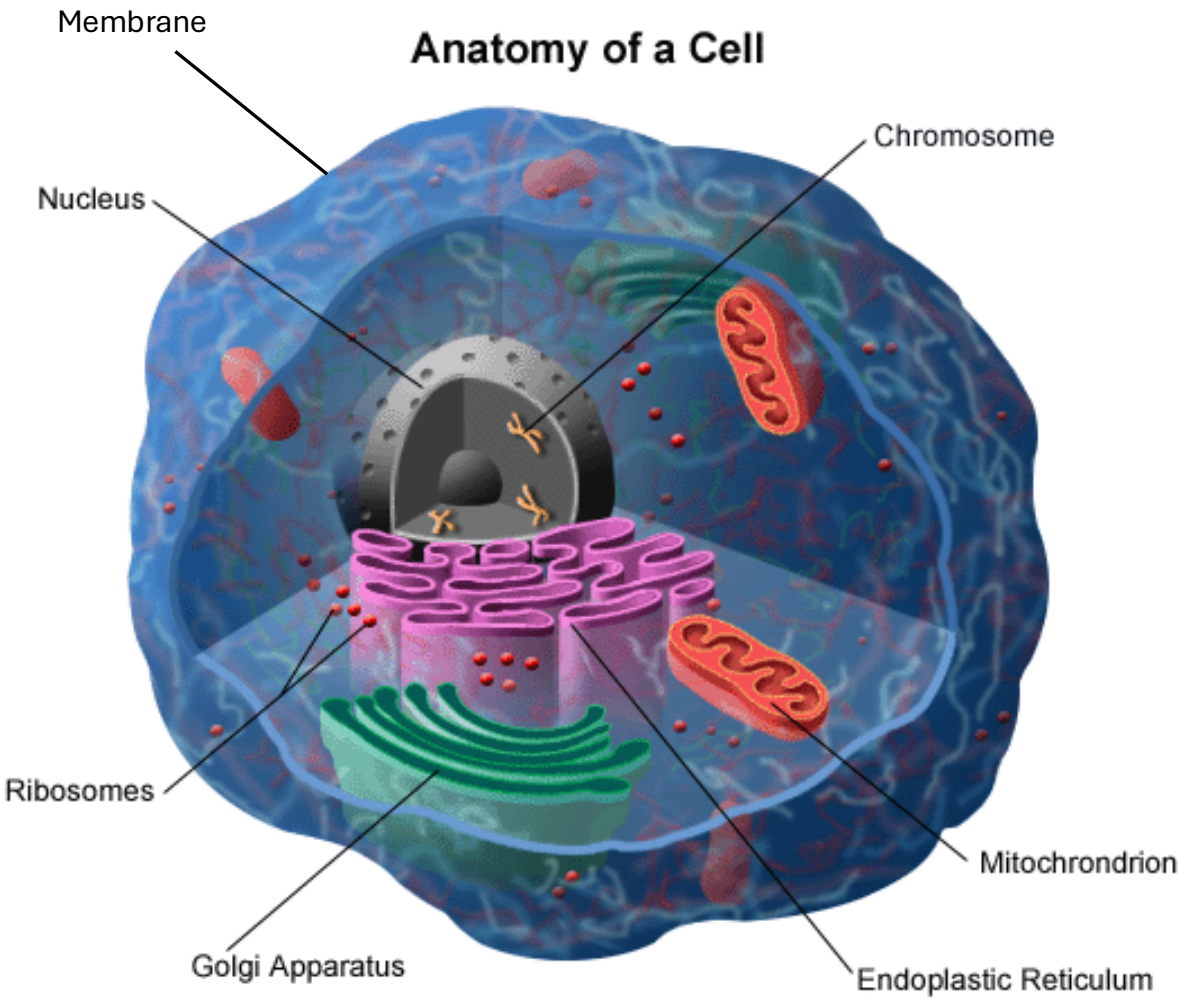
- Individuals of a population that are fitter tend to survive for longer and reproduce
- Their characteristics, encoded in their genes, are transmitted to their offspring and, thus, tend to propagate into future generations
- In sexual reproduction, the genes of the offspring are a mix of those of their parents.
- Offspring's characteristics are partially inherited from their parents, and partially the result of new genes created during the process of reproduction.

Natural selection does not quite mean ”*the survival of the fittest*”

- “Fittest” means “*best adapted*”, not “in the best condition”
- “Best adapted” means “best adapted with respect to a *niche*” (a relevant sub-set of the environment)
- “Length of life” does not mean “*fertility*”
- “Fertility” does not mean “*successful reproduction*” (i.e., production of offspring who live long enough to reproduce themselves)

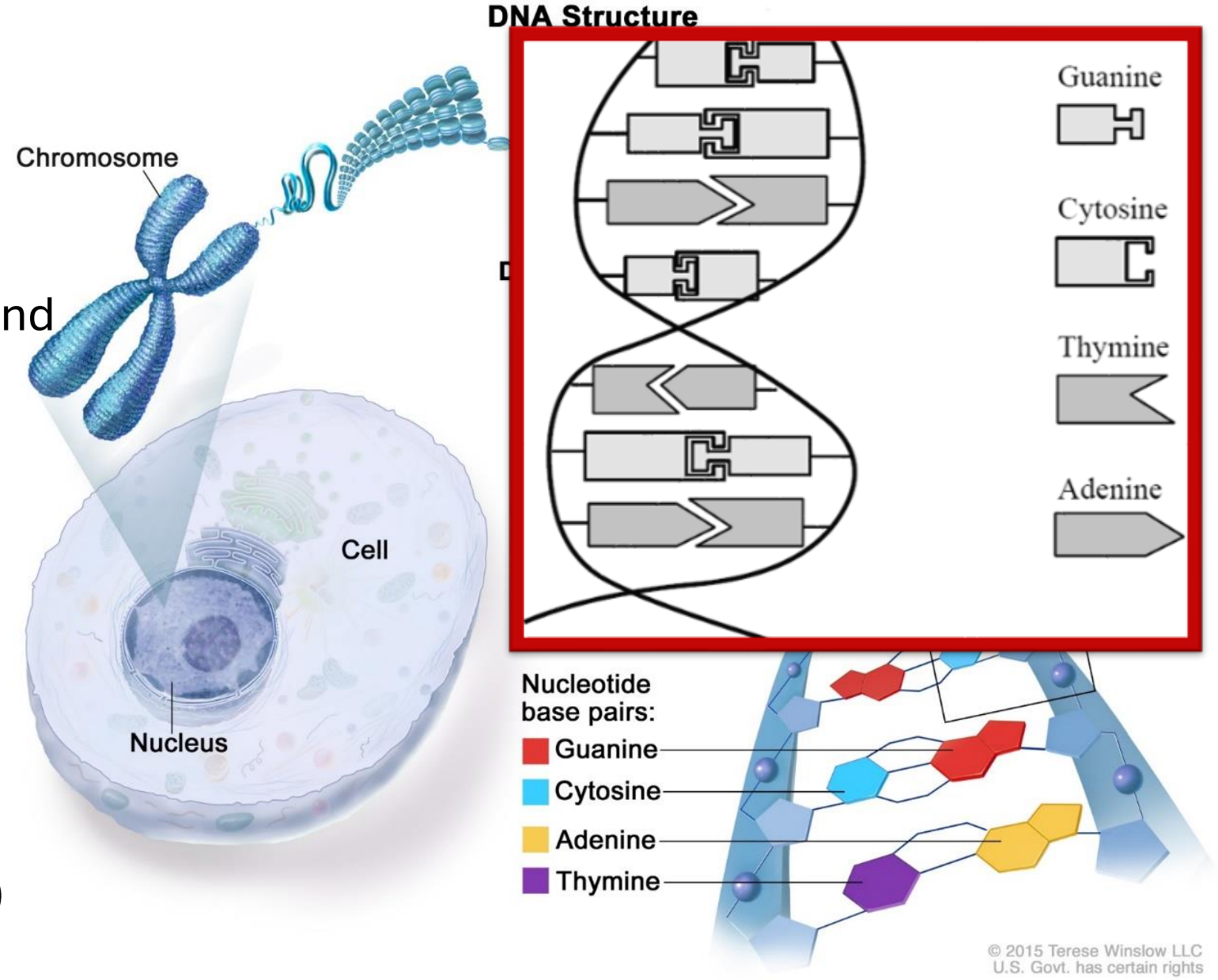
The cell

- Cells are surrounded by a thin oily **membrane**
- The membrane has a **voltage** of about 100mV across caused by different concentrations of ions inside and outside
- Cells include a **nucleus**
- The nucleus includes several **chromosomes** (“coloured bodies”)
- In diploid cells, chromosomes are **paired** (homologous chromosomes), the two elements of a pair carrying similar information
- Chromosomes are made up of **DNA** (deoxyribonucleic acid)



DNA (1)

- DNA is made up of **bases** and other molecules
- DNA **bases** are:
  - **A** (adenine)
  - **T** (thymine)
  - **C** (cytosine) and
  - **G** (guanine)
- These molecules are organised to form a double helix (spiral-shaped ladder)



https://siteman.wustl.edu/wp-content/uploads/ncipdq-media/CDR0000761781.jpg

DNA (2)

- **DNA** is a long description (a book) whose "**characters**" are T, G, C, A (about 3 billion bases in humans)
- The "**words**" of the DNA include three bases (**triplets** or **codons**) and represent **amino-acids** (building blocks of proteins)
  - e.g., TCT=Serine, CAA=Glutamine, TAA=Stop
- A **gene** is represented by a "**sentence**" (a *meaningful* sequence of triplets) in the DNA
- Some triplets represent **stop** symbols (the end of a sentence)

Genetic code (1)

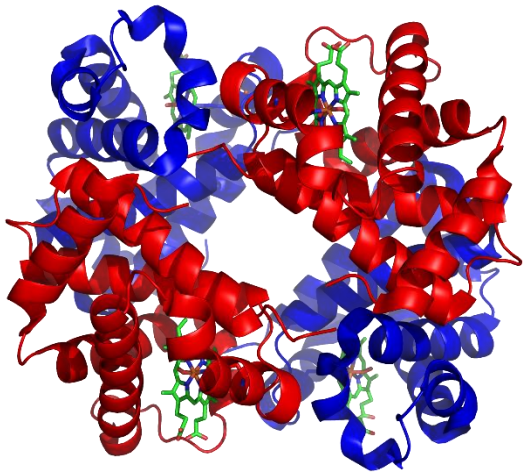
First Letter	Second Letter: T	Second Letter: C	Second Letter: A	Second Letter: G	Third Letter
T	TTT, TTC: Phe TTA, TTG: Leu	TCT, TCC, TCA, TCG: Ser	TAT, TAC: Tyr TAA: Stop TAG: Stop	TGT, TGC: Cys TGA: Stop TGG: Trp	T C A G
C	CTT, CTC, CTA, CTG: Leu	CCT, CCC, CCA, CCG: Pro	CAT, CAC: His CAA, CAG: Gln	CGT, CGC, CGA, CGG: Arg	T C A G
A	ATT, ATC, ATA: Ile ATG: Met	ACT, ACC, ACA, ACG: Thr	AAT, AAC: Asn AAA, AAG: Lys	AGT, AGC: Ser AGA, AGG: Arg	T C A G
G	GTT, GTC, GTA, GTG: Val	GCT, GCC, GCA, GCG: Ala	GAT, GAC: Asp GAA, GAG: Glu	GGT, GGC, GGA, GGG: Gly	T C A G

Also start symbol

Genetic code (2)

- Most genes (sentences in the DNA) **code for proteins** (**structural** genes, i.e., trait determining factors)

E.g., haemoglobin→



- Some segments of the DNA have **regulatory** purposes, i.e., they bind to specific sequences in the DNA and control which **genes to turn on or off** under any particular **conditions**.
- Other genes have **no function** (*introns*, ”*junk DNA*”) or we do **not understand** their function yet.

Genetic code (3)

- A gene can be represented by a number of slightly different sequences called **alleles**.
  - **Gene** is a portion of the DNA that determines a **trait**
  - **Allele** express the **characteristics** (e.g., hair colour)
- Each homologous chromosome may have a different allele for each gene (remember the peas example)
- Approximately speaking, alleles are **dominant** if they **produce much more** (and sufficient) **protein** than their recessive counterparts.

Genetic code (4)

- **Central dogma** of molecular biology (*DNA as blueprint*):
  - DNA **replicates** to produce more DNA (*DNA→DNA*)
  - DNA is **transcribed** into ribonucleic acid (**RNA**) which is translated into **proteins** (*DNA→RNA→Protein*)
- During these processes DNA can undergo **mutations**
  - **Copy** error (10^{-10} nucleotides)
  - Induced due to **exposure** to chemicals, UV, X-rays, … (**external** influences)
  - **Spontaneous** due to reactions within organism

Type of mutations

- **Substitution**: Exchange one base for another
  - Condon (triplet) encodes a different amino acid, the same amino acid or STOP (incomplete proteins)
- **Insertion**: Extra base pair inserted
- **Deletion**: Section of DNA lost or deleted
- **Frameshift**: Insertions/deletions can alter a gene so that message is no longer correctly parsed.

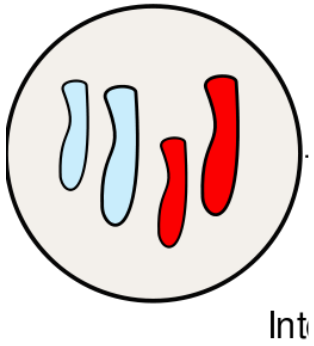
  Example: The fat cat sat → hef atc ats at
- …
- Mutation can be **beneficial**, **neutral**, or **harmful**!

Summary of main properties of DNA

- It **stores** and **transmits** information
- It **copies** itself (mainly) to generate proteins but also to transmit information
- It can **mutate**

Meiosis – Reproduction (1)

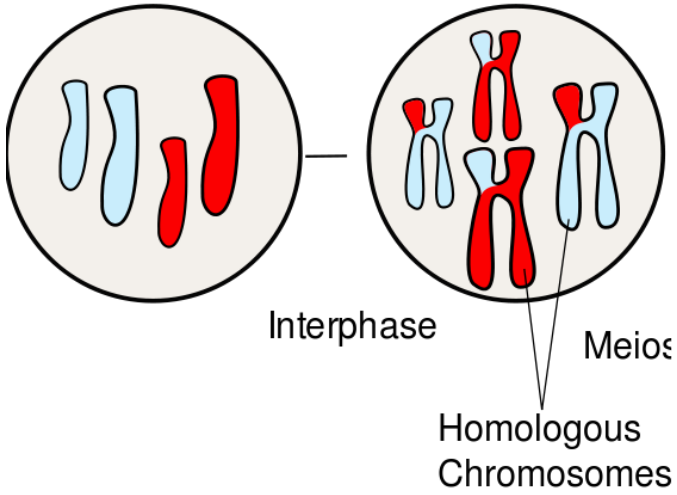
In **meiosis** (preparation of egg and sperm cells) **homologous chromosomes** duplicate (46→92) (cyan=maternal, red=paternal)



Shyamala Iyer. (2014, February 03). Cell Division. ASU - Ask A Biologist. Retrieved January 8, 2020 from https://askabiologist.asu.edu/cell-division

Meiosis – Reproduction (2)

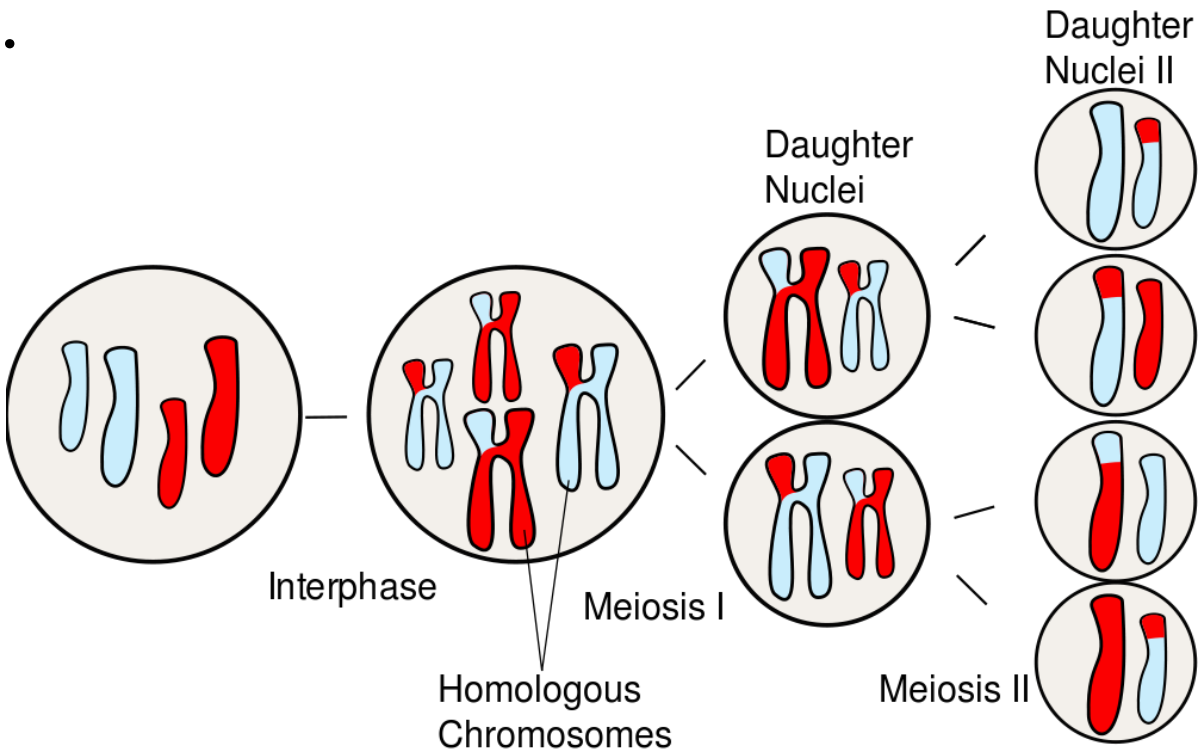
Because of **close proximity**, they can become entangled, resulting in chromosomes that are a **mixture** of maternal and paternal genes (“**crossover**" or "**genetic recombination**”)



Shyamala Iyer. (2014, February 03). Cell Division. ASU - Ask A Biologist. Retrieved January 8, 2020 from https://askabiologist.asu.edu/cell-division

Meiosis – Reproduction (3)

- The cell then divides twice so that only one of the homologous chromosomes of a pair is present in the resulting cells (sperm or eggs).



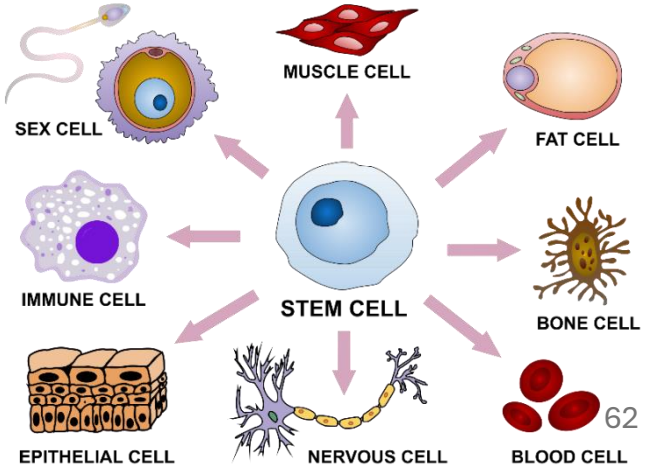
Shyamala Iyer. (2014, February 03). Cell Division. ASU - Ask A Biologist. Retrieved January 8, 2020 from https://askabiologist.asu.edu/cell-division

Mating and Growth

- Mating produces (**diploid**) cells that start duplicating (**mitosis**)
- Duplication involves chromosome copying and cell division (**no crossover**). **Mutations** can occur during the **copying** phase.
- The process of growth transforms the information in the genes (**genotype**) into an adult individual (**phenotype**).
- Growth is controlled mainly by the **environment** (but also by genetic factors)
- During the development of an individual **cells specialise** (differentiation) and **migrate**

Cell differentiation

- **Wait: all cells have exactly the same DNA!** So, how do we get skin cells, neurons, muscle cells, …?
- Cell **differentiation** happens thanks to the **regulation of gene expression** determined by environment surrounding the cell
- Regulation is performed by tracts of DNA that function as switches for groups of genes.



By Haileyfournier - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=79600426

Questions?

What did we learn today? (1)

- We have learned that **combinatorial optimisation** problems (Knapsack problem & Bin Packing problem) can be solved with **meta-heuristic stochastic optimisation methods** such as **Evolutionary Algorithms**.
- In this module we will learn about EAs and work with them as tools to solve different types of (continuous and combinatorial) optimisation problems.
- EAs can also be used a form of machine learning: program induction (**Genetic Programming**)
- We also learned what is required in terms of assessment.

What did we learn today? (2)

- **Natural selection** means individuals in a population that are fitter with respect to their environment produce more offspring. The trait that made those individuals succeed becomes common in the population.
- **Genes** encode the various traits of an individual. All traits of an individual are combined in a long **linear representation** (1-D array) (DNA).
- **Genetic recombination (crossover)**, i.e., the exchange of genetic material to create offspring's that are different than the parents, and **random mutations** drive evolution.

Don’t forget to …

…catch-up with Python coding if necessary in preparation for the lab this Friday.

Resources:

- Python (https://www.python.org/)
- Jupyter Lab (https://jupyter.org/)
- Anaconda (https://www.anaconda.com/)
- Google Colab (https://colab.research.google.com)
- VirtualLab (https://csee-horizon.essex.ac.uk/)

[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Thanks for your participation!
Next time:
Translating natural evolution into algorithms

University of Essex
School of Computer Science and Electronic Engineering